

Stage 0	Stage 1	Stage 2	Stage 3	Stage 4
ABCDE	(AB)CDE A(BC)DE AB(CD)E ABC(DE)	(ABC)DE (AB)(CD)E (AB)C(DE) A(BCD)E A(BC)(DE) AB(CDE)	(ABCD)E (ABC)(DE) (AB)(CDE) A(BCDE)	(ABCDE)

Here parentheses group matrix factors that are already fully multiplied (i.e., they are, by now, a single matrix). We will leave it to you to connect the states by allowed edges, and to calculate the cost of each edge in terms of the dimensions of the various matrices.

So how could we have done this example *wrong*? We might have identified states with all possible ways of parenthesizing **ABCDE**, including, for example, **A(B(CD)E)**. That is unnecessary, because only the outermost parentheses matter: A matrix state doesn't care about the exact path taken to reach it, as long as its factors were multiplied in some associative order. The power of DP is realized when, at every stage, many histories collapse to a (relatively) small number of states, which can then be taken as ignorant of their past history.

10.13.2 Example: DNA Sequence Alignment

DNA sequences of different organisms, at one time identical in a common ancestor, can diverge over time by the deletion, insertion, or substitution of bases in one or the other organism's sequence. It is desired to find the best match between two given sequences. In finding the best match, we are allowed to insert gaps in either sequence; but in the end we will be assessed a penalty for gap positions, a penalty for mismatches, and a reward for matches.

For example [2], before matching, we might have the two sequences

```

G  A  A  T  T  C  A  G  T  T  A
G  G  A  T  C  G  A

```

A possible match might be

```

G  A  A  T  T  C  A  G  T  T  A
G  G  A      T  C      G      A

```

for which we would earn six rewards, less one mismatch penalty (shown *italic*) and four gap penalties. (We will consider all rewards and penalties as having positive or zero values, with the accounting done by *adding* rewards and *subtracting* penalties.)

Needleman and Wunsch [1] first pointed out that this problem is amenable to solution by DP, allowing *all possible* matchings to be scored, and the highest-scoring one identified. The clever idea is to form a two-dimensional array with the two sequences defining the columns and rows. In the above example, this looks like

		G	A	A	T	T	C	A	G	T	T	A
	0											
G												
G												
A												
T												
C												
G												
A												

A matching consists of a path through the (initially) empty boxes in the above tableau, starting in the box labeled by a zero and moving, in each step, either one box right, one box down, or diagonally down and right. A right or down move corresponds to using up a letter in the first or second sequence (respectively) without using up one in the second or first (respective) sequence. Therefore it corresponds to inserting a gap and incurs a gap penalty. A diagonal move corresponds to pairing a new character in each sequence. Therefore, it incurs either the matching reward, if the two sequences match, or a mismatch penalty, if they don't.

Also useful is to distinguish right or down moves between two boxes in the bordering rows or columns (respectively) of the table from right or down moves in the interior of the table. The former kind don't open gaps in either sequence, but merely allow the sequences to shift with respect to one another. So we might assess a smaller, or zero, penalty for these overall "skews."

Now, characteristic of dynamic programming, you just fill in the boxes with the total score of the *best* way to reach that box, either from above it, from its left, or from the upper-left diagonal neighbor. A score is computed, naturally, by taking the score of a predecessor box and then adding the reward (or subtracting the penalty) associated with the move. Starting at the upper left, there are always boxes ready to be filled in as you work your way, by rows or columns, to the lower right.

Also characteristic of dynamic programming, when you are done filling in all the boxes, you do a backtrack pass: Start at the lower right box in the table. Now figure out the path back through the table that contributed to that best score. (It may not be unique.) Finally, translate that path into the series of letters and gaps that it implies.

A straightforward implementation of this is the following routine. Note that the matching reward is normalized to one per matched character, while the penalties for mismatches, gaps, and overall skews are input arguments. You can set all three to zero in most cases. If you set a nonzero mismatch penalty, however, you will probably also want to have a gap penalty, since otherwise the program will always avoid a mismatch by creating two gaps, one in each string.

The routine's output includes a summary string showing where the matches, mismatches, and gaps occur. For the example above, the summary string is

= != == = =

(You can change the symbols used to your liking.)

You can modify the program in various other ways. For example, you might want to have a bigger penalty for initially opening a gap than for extending it once

opened. This requires a more complicated logic in the initial filling of the cost table. As long as you are able to fill each box with the cost of the *best* way to reach it, then the logic of dynamic programming will still apply.

```
void stringalign(char *ain, char *bin, Doub mispen, Doub gappen,          stringalign.h
                Doub skwpen, char *aout, char *bout, char *summary) {
    Given null terminated input strings ain and bin, and given penalties mispen, gappen, and
    skwpen, respectively, for mismatches, interior gaps, and gaps before/after either string, set null
    terminated output strings aout, bout, and summary as the aligned versions of the input strings,
    and a summary string. User must supply storage for the output strings of size equal to the sum
    of the two input strings.
    Int i,j,k;
    Doub dn,rt,dg;                                                    Cost of down, right, and diagonal moves.
    Int ia = strlen(ain), ib = strlen(bin);
    MatDoub cost(ia+1,ib+1);                                          Cost table, as illustrated in the text.
    First we fill in the cost table.
    cost[0][0] = 0.;
    for (i=1;i<=ia;i++) cost[i][0] = cost[i-1][0] + skwpen;
    for (i=1;i<=ib;i++) cost[0][i] = cost[0][i-1] + skwpen;
    for (i=1;i<=ia;i++) for (j=1;j<=ib;j++) {
        dn = cost[i-1][j] + ((j == ib)? skwpen : gappen);
        rt = cost[i][j-1] + ((i == ia)? skwpen : gappen);
        dg = cost[i-1][j-1] + ((ain[i-1] == bin[j-1])? -1. : mispen);
        cost[i][j] = MIN(MIN(dn,rt),dg);
    }
    Next, we do the backtrack pass, writing the output (backward, however).
    i=ia; j=ib; k=0;
    while (i > 0 || j > 0) {
        dn = rt = dg = 9.99e99;                                        Any large value will do.
        if (i>0) dn = cost[i-1][j] + ((j==ib)? skwpen : gappen);
        if (j>0) rt = cost[i][j-1] + ((i==ia)? skwpen : gappen);
        if (i>0 && j>0) dg = cost[i-1][j-1] +
            ((ain[i-1] == bin[j-1])? -1. : mispen);
        if (dg <= MIN(dn,rt)) {                                       Diagonal move produces either match or in-
            aout[k] = ain[i-1];                                       equality.
            bout[k] = bin[j-1];
            summary[k++] = ((ain[i-1] == bin[j-1])? '=' : '!');
            i--; j--;
        }
        else if (dn < rt) {                                           Down move produces a gap in the B string.
            aout[k] = ain[i-1];
            bout[k] = ' ';
            summary[k++] = ' ';
            i--;
        }
        else {                                                        Right move produces a gap in the A string.
            aout[k] = ' ';
            bout[k] = bin[j-1];
            summary[k++] = ' ';
            j--;
        }
    }
    Finally, reverse the output strings.
    for (i=0;i<k/2;i++) {
        SWAP(aout[i],aout[k-1-i]);
        SWAP(bout[i],bout[k-1-i]);
        SWAP(summary[i],summary[k-1-i]);
    }
    aout[k] = bout[k] = summary[k] = 0;                               Don't forget the terminating nulls!
}
```

Various modifications of the Needleman-Wunsch method are also in use, most

notably the generalization by Smith and Waterman [3]. There are also a number of heuristic methods for identifying sequence similarity, with names like BLAST, FASTA, BLAT, etc. The field is highly developed, so you should use the routine above only pedagogically.

CITED REFERENCES AND FURTHER READING:

- Cybenko, G. 1997, "Dynamic Programming: A Discrete Calculus of Variations," *IEEE Computational Science and Engineering*, vol. 4, no. 1, pp. 92-97.
- Bertsekas, 2001, *Dynamic Programming and Optimal Control*, 2nd ed., 2 vols. (Belmont, MA: Athena Scientific).
- Hillier, F.S. and Lieberman, G.J. 2002, *Introduction to Operations Research*, 7th ed. (New York: McGraw-Hill).
- Needleman, S.B. and Wunsch, C.D. 1970, "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins," *Journal of Molecular Biology*, vol. 48, pp. 443-453.[1]
- Rouchka, E.C. 2001, "Dynamic Programming," at multiple Web sites.[2]
- Smith, T.F. and Waterman, M.S. 1981, "Identification of Common Molecular Subsequences," *Journal of Molecular Biology*, vol. 147, pp. 195-197.[3]

11.0 Introduction

An $N \times N$ matrix $\mathbf{A}$ is said to have an *eigenvector* $\mathbf{x}$ and corresponding *eigenvalue* λ if

$$\mathbf{A} \cdot \mathbf{x} = \lambda \mathbf{x} \quad (11.0.1)$$

Obviously any multiple of an eigenvector $\mathbf{x}$ will also be an eigenvector, but we won't consider such multiples as being distinct eigenvectors. (The zero vector is not considered to be an eigenvector at all.) Evidently (11.0.1) can hold only if

$$\det |\mathbf{A} - \lambda \mathbf{I}| = 0 \quad (11.0.2)$$

which, if expanded out, is an N th degree polynomial in λ whose roots are the eigenvalues. This proves that there are always N (not necessarily distinct) eigenvalues. Equal eigenvalues coming from multiple roots are called *degenerate*. Root searching in the characteristic equation (11.0.2) is usually a very poor computational method for finding eigenvalues. We will learn much better ways in this chapter, as well as efficient ways for finding corresponding eigenvectors.

The above two equations also prove that every one of the N eigenvalues has a (not necessarily distinct) corresponding eigenvector: If λ is set to an eigenvalue, then the matrix $\mathbf{A} - \lambda \mathbf{I}$ is singular, and we know that every singular matrix has at least one nonzero vector in its nullspace (see §2.6.1).

If you add $\tau \mathbf{x}$ to both sides of (11.0.1), you will easily see that the eigenvalues of any matrix can be changed or *shifted* by an additive constant τ by adding to the matrix that constant times the identity matrix. The eigenvectors are unchanged by this shift. Shifting, as we will see, is an important part of many algorithms for computing eigenvalues. We see also that there is no special significance to a zero eigenvalue. Any eigenvalue can be shifted to zero, or any zero eigenvalue can be shifted away from zero.

11.0.1 Definitions and Basic Facts

A matrix is called *symmetric* if it is equal to its transpose,

$$\mathbf{A} = \mathbf{A}^T \quad \text{or} \quad a_{ij} = a_{ji} \quad (11.0.3)$$

It is called *Hermitian* or *self-adjoint* if it equals the complex conjugate of its transpose (its *Hermitian conjugate*, denoted by “†”)

$$\mathbf{A} = \mathbf{A}^\dagger \quad \text{or} \quad a_{ij} = a_{ji}^* \quad (11.0.4)$$

It is termed *orthogonal* if its transpose equals its inverse,

$$\mathbf{A}^T \cdot \mathbf{A} = \mathbf{A} \cdot \mathbf{A}^T = \mathbf{1} \quad (11.0.5)$$

and *unitary* if its Hermitian conjugate equals its inverse. Finally, a matrix is called *normal* if it *commutes* with its Hermitian conjugate,

$$\mathbf{A} \cdot \mathbf{A}^\dagger = \mathbf{A}^\dagger \cdot \mathbf{A} \quad (11.0.6)$$

For real matrices, Hermitian means the same as symmetric, unitary means the same as orthogonal, and *both* of these distinct classes are normal.

The reason that Hermitian is an important concept has to do with eigenvalues. The eigenvalues of a Hermitian matrix are all real. In particular, the eigenvalues of a real symmetric matrix are all real. Contrariwise, the eigenvalues of a real nonsymmetric matrix may include real values, but may also include pairs of complex-conjugate values; and the eigenvalues of a complex matrix that is not Hermitian will in general be complex.

The reason that normal is an important concept has to do with the eigenvectors. The eigenvectors of a normal matrix with nondegenerate (i.e., distinct) eigenvalues are complete and orthogonal, spanning the N -dimensional vector space. For a normal matrix with degenerate eigenvalues, we have the additional freedom of replacing the eigenvectors corresponding to a degenerate eigenvalue by linear combinations of themselves. Using this freedom, we can always perform Gram-Schmidt orthogonalization (consult any linear algebra text) and *find* a set of eigenvectors that are complete and orthogonal, just as in the nondegenerate case. The matrix whose columns are an orthonormal set of eigenvectors is evidently unitary. A special case is that the matrix of eigenvectors of a real symmetric matrix is orthogonal, since the eigenvectors of that matrix are all real.

When a matrix is not normal, as typified by any random, nonsymmetric, real matrix, then in general we cannot find *any* orthonormal set of eigenvectors, nor even any pairs of eigenvectors that are orthogonal (except perhaps by rare chance). While the N nonorthonormal eigenvectors will “usually” span the N -dimensional vector space, they do not always do so; that is, the eigenvectors are not always complete. Such a matrix is said to be *defective*.

11.0.2 Left and Right Eigenvectors

While the eigenvectors of a nonnormal matrix are not particularly orthogonal among themselves, they *do* have an orthogonality relation with a different set of vectors, which we must now define. Up to now our eigenvectors have been column vectors that are multiplied to the right of a matrix $\mathbf{A}$, as in (11.0.1). These, more explicitly, are termed *right eigenvectors*. We could also, however, try to find row vectors, which multiply $\mathbf{A}$ to the left and satisfy

$$\mathbf{x} \cdot \mathbf{A} = \lambda \mathbf{x} \quad (11.0.7)$$

These are called *left eigenvectors*. By taking the transpose of equation (11.0.7), we see that every left eigenvector is the transpose of a right eigenvector *of the transpose of A*. Now, by comparing to (11.0.2) and using the fact that the determinant of a matrix equals the determinant of its transpose, we also see that the left and right *eigenvalues* of **A** are identical.

If the matrix **A** is symmetric, then the left and right eigenvectors are just transposes of each other, that is, they have the same numerical components. Likewise, if the matrix is self-adjoint, the left and right eigenvectors are Hermitian conjugates of each other. For the general nonnormal case, however, we have the following calculation: Let $\mathbf{X}_R$ be the matrix formed by columns from the right eigenvectors and $\mathbf{X}_L$ be the matrix formed by rows from the left eigenvectors. Then (11.0.1) and (11.0.7) can be rewritten as

$$\mathbf{A} \cdot \mathbf{X}_R = \mathbf{X}_R \cdot \text{diag}(\lambda_0 \dots \lambda_{N-1}) \quad \mathbf{X}_L \cdot \mathbf{A} = \text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_L \quad (11.0.8)$$

Multiplying the first of these equations on the left by $\mathbf{X}_L$, the second on the right by $\mathbf{X}_R$, and subtracting the two, gives

$$(\mathbf{X}_L \cdot \mathbf{X}_R) \cdot \text{diag}(\lambda_0 \dots \lambda_{N-1}) = \text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot (\mathbf{X}_L \cdot \mathbf{X}_R) \quad (11.0.9)$$

This says that the matrix of dot products of the left and right eigenvectors commutes with the diagonal matrix of eigenvalues. But the only matrices that commute with a diagonal matrix *of distinct elements* are themselves diagonal. Thus, if the eigenvalues are nondegenerate, each left eigenvector is orthogonal to all right eigenvectors except its corresponding one, and vice versa. By choice of normalization, the dot products of corresponding left and right eigenvectors can always be made unity for any matrix with nondegenerate eigenvalues.

If some eigenvalues are degenerate, then either the left or the right eigenvectors corresponding to a degenerate eigenvalue must be linearly combined among themselves to achieve orthogonality with the right or left ones, respectively. This can always be done by a procedure akin to Gram-Schmidt orthogonalization. The normalization can then be adjusted to give unity for the nonzero dot products between corresponding left and right eigenvectors. If the dot product of corresponding left and right eigenvectors is zero at this stage, then you have a case where the eigenvectors are incomplete! Note that incomplete eigenvectors can occur only where there are degenerate eigenvalues, but they do not always occur in such cases (in fact, they never occur for the class of “normal” matrices). See [1] for a clear discussion.

In both the degenerate and nondegenerate cases, the final normalization to unity of all nonzero dot products produces the result: The matrix whose rows are left eigenvectors is the inverse matrix of the matrix whose columns are right eigenvectors, *if the inverse exists*. When it does exist, equations (11.0.8) and (11.0.9) imply the useful factorizations

$$\mathbf{A} = \mathbf{X}_R \cdot \text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{X}_L \quad \text{and} \quad \text{diag}(\lambda_0 \dots \lambda_{N-1}) = \mathbf{X}_L \cdot \mathbf{A} \cdot \mathbf{X}_R \quad (11.0.10)$$

11.0.3 Diagonalization of a Matrix

From equation (11.0.10) and the fact that $\mathbf{X}_L$ and $\mathbf{X}_R$ are matrix inverses, we get

$$\mathbf{X}_R^{-1} \cdot \mathbf{A} \cdot \mathbf{X}_R = \text{diag}(\lambda_0 \dots \lambda_{N-1}) \quad (11.0.11)$$